**EX. NO: 5** **CIRCULAR SINGLY LINKED LISTS**

**DATE:**

**QUESTION :**

**Implement creation of a sorted linked list, Deletion of a node, search a value, merge two sorted lists, and display operations on Circular Singly Linked List.**

**AIM:**

To implement creation, deletion, searching, merging, and display operations on a Circular Singly Linked List.

**ALGORITHM:**

Step-1. Create Sorted Circular Linked List

- Start.
- Create a new node and store the given value.
- If the list is empty, make the new node point to itself and make it the head.
- If the value is smaller than the head value, insert the node before the head.
- Otherwise, traverse the list until the correct sorted position is found.
- Insert the new node at that position.
- Stop.

Step-2. Display

- If the list is empty, display "List is Empty".
- Otherwise, start from head.
- Display each node and move to the next node.
- Stop when the pointer reaches head again.

Step-3. Search

- If the list is empty, display "List Empty".
- Traverse the circular list from head.
- Compare each node's data with the search value.
- If found, display "Element Found".
- If traversal reaches head again, display "Element Not Found".

Step-4. Delete a Node

- If the list is empty, display "List Empty".
- Search for the node containing the given value.
- If it is the only node, delete it and set head = NULL.
- If it is the head node, update the last node's link and change head.
- Otherwise, connect the previous node to the next node and delete the target node.
- Stop.

Step-5. Merge Two Sorted Lists

- Create an empty result list.
- Traverse the first circular list and insert each element into the result list in sorted order.
- Traverse the second circular list and insert each element into the result list in sorted order.
- Return the merged circular sorted list.
- Stop.

Step-6. Main Operation

- Display the menu.
- Read the user's choice.
- Perform create, display, delete, search, or merge operation according to the choice.
- Repeat until the user selects Exit.
- Stop.

**PROGRAM:**

```
#include <stdio.h>
#include <stdlib.h>
struct Node{
   int data;
   struct Node *next; };
struct Node *head1 = NULL, *head2 = NULL;
void insertSorted(struct Node **head, int value) {
   struct Node *newNode, *current;
   newNode = (struct Node *)malloc(sizeof(struct Node));
   newNode->data = value;
   if (*head == NULL) {
      newNode->next = newNode;
      *head = newNode;
      return; }
   if (value < (*head)->data) {
      current = *head;
      while (current->next != *head)
         current = current->next;
      current->next = newNode;
      newNode->next = *head;
      *head = newNode;
      return; }
   current = *head;
   while (current->next != *head && current->next->data < value)
      current = current->next;
   newNode->next = current->next;
   current->next = newNode; }
void display(struct Node *head) {
   if (head == NULL) {
      printf("List is Empty\n");
      return;
   }
```

```
    struct Node *temp = head;
    do
    {
        printf("%d ", temp->data);
        temp = temp->next;
    } while (temp != head);
    printf("\n"); }
void search(struct Node *head, int key) {
    if (head == NULL) {
        printf("List Empty\n");
        return;
    }
    struct Node *temp = head;
    do
    {
        if (temp->data == key)
        {
            printf("Element Found\n");
            return;
        }
        temp = temp->next;
    } while (temp != head);
    printf("Element Not Found\n");
}
void deleteNode(struct Node **head, int key)
{
    if (*head == NULL)
    {
        printf("List Empty\n");
        return;
    }
    struct Node *curr = *head, *prev = NULL;
    if (curr->data == key)
    {
        if (curr->next == *head)
        {
            free(curr);
            *head = NULL;
            return;
        }
        while (curr->next != *head)
            curr = curr->next;
```

```
        struct Node *temp = *head;
        curr->next = temp->next;
        *head = temp->next;
        free(temp);
        printf("Deleted Successfully\n");
        return;
    }
    curr = *head;
    while (curr->next != *head && curr->data != key)
    {
        prev = curr;
        curr = curr->next;
    }
    if (curr->data == key)
    {
        prev->next = curr->next;
        free(curr);
        printf("Deleted Successfully\n");
    }
    else
        printf("Element Not Found\n");
}
struct Node *merge(struct Node *head1, struct Node *head2)
{
    struct Node *result = NULL;
    if (head1 != NULL)
    {
        struct Node *temp = head1;
        do
        {
            insertSorted(&result, temp->data);
            temp = temp->next;
        } while (temp != head1);
    }
    if (head2 != NULL)
    {
        struct Node *temp = head2;
        do
        {
            insertSorted(&result, temp->data);
            temp = temp->next;
        } while (temp != head2);
```

```
    }
    return result;
}
int main()
{
    int n, i, value, choice, key;
    struct Node *merged = NULL;
    while (1)
    {
        printf("\n--- Circular Singly Linked List ---\n");
        printf("1.Create List1\n");
        printf("2.Create List2\n");
        printf("3.Display List1\n");
        printf("4.Display List2\n");
        printf("5.Delete from List1\n");
        printf("6.Search in List1\n");
        printf("7.Merge Lists\n");
        printf("8.Display Merged List\n");
        printf("9.Exit\n");
        printf("Enter Choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
        case 1:
            printf("Enter number of elements: ");
            scanf("%d", &n);
            for (i = 0; i < n; i++)
            {
                scanf("%d", &value);
                insertSorted(&head1, value);
            }
            break;
        case 2:
            printf("Enter number of elements: ");
            scanf("%d", &n);
            for (i = 0; i < n; i++)
            {
                scanf("%d", &value);
                insertSorted(&head2, value);
            }
            break;
```

```
        case 3:
            printf("List1: ");
            display(head1);
            break;
        case 4:
            printf("List2: ");
            display(head2);
            break;
        case 5:
            printf("Enter element to delete: ");
            scanf("%d", &key);
            deleteNode(&head1, key);
            break;
        case 6:
            printf("Enter element to search: ");
            scanf("%d", &key);
            search(head1, key);
            break;
        case 7:
            merged = merge(head1, head2);
            printf("Lists Merged Successfully\n");
            break;
        case 8:
            printf("Merged List: ");
            display(merged);
            break;
        case 9:
            exit(0);
        default:
            printf("Invalid Choice\n");
        }
    }
    return 0;
}
```

**INPUT AND OUTPUT:**

```
[ita2536@mritlin harshan]$ vi ex5.c
[ita2536@mritlin harshan]$ cc ex5.c
[ita2536@mritlin harshan]$ ./a.out
--- Circular Singly Linked List ---
1.Create List1
2.Create List2
3.Display List1
4.Display List2
5.Delete from List1
6.Search in List1
7.Merge Lists
8.Display Merged List
9.Exit
Enter Choice: 1
Enter number of elements: 3
1
2
3

--- Circular Singly Linked List ---
1.Create List1
2.Create List2
3.Display List1
4.Display List2
5.Delete from List1
6.Search in List1
7.Merge Lists
8.Display Merged List
9.Exit
Enter Choice: 2
Enter number of elements: 3
2
4
6

--- Circular Singly Linked List ---
1.Create List1
2.Create List2
3.Display List1
4.Display List2
5.Delete from List1
6.Search in List1
7.Merge Lists
8.Display Merged List
9.Exit
Enter Choice: 3
List1: 1 2 3
```

```
--- Circular Singly Linked List ---
1.Create List1
2.Create List2
3.Display List1
4.Display List2
5.Delete from List1
6.Search in List1
7.Merge Lists
8.Display Merged List
9.Exit
Enter Choice: 4
List2: 2 4 6

--- Circular Singly Linked List ---
1.Create List1
2.Create List2
3.Display List1
4.Display List2
5.Delete from List1
6.Search in List1
7.Merge Lists
8.Display Merged List
9.Exit
Enter Choice: 5
Enter element to delete: 1
Deleted Successfully

--- Circular Singly Linked List ---
1.Create List1
2.Create List2
3.Display List1
4.Display List2
5.Delete from List1
6.Search in List1
7.Merge Lists
8.Display Merged List
9.Exit
Enter Choice: 6
Enter element to search: 2
Element Found
```

```
--- Circular Singly Linked List ---
1.Create List1
2.Create List2
3.Display List1
4.Display List2
5.Delete from List1
6.Search in List1
7.Merge Lists
8.Display Merged List
9.Exit
Enter Choice: 7
Lists Merged Successfully

--- Circular Singly Linked List ---
1.Create List1
2.Create List2
3.Display List1
4.Display List2
5.Delete from List1
6.Search in List1
7.Merge Lists
8.Display Merged List
9.Exit
Enter Choice: 8
Merged List: 2 2 3 4 6

--- Circular Singly Linked List ---
1.Create List1
2.Create List2
3.Display List1
4.Display List2
5.Delete from List1
6.Search in List1
7.Merge Lists
8.Display Merged List
9.Exit
Enter Choice: 9
[ita2536@mritlin harshan]$
```

**RESULT:**

Thus, the implementation of Circular Singly Linked List with sorted creation, deletion, searching, merging, and display operations was executed successfully.